UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Computación Paralela y Distribuida



Proyecto #3

Detección de rectas con la Transformada de Hough en CUDA

Integrantes:

Erick Guerra

Diego Fabián Morales

Juan Cruz

Guatemala, 11 de noviembre del 2026

https://github.com/cru23110/proyecto3-cuda-hough

Índice

1 Introducción 1

2 Antecedentes de la Transformada de Hough 1
- 2.1 La Transformada lineal de Hough 1
- 2.2 Coordenadas centradas 3
- 2.3 Selección de rectas 3
- 2.4 Jerarquía de memoria de la GPU 4

3 Descripción de los programas 5
- 3.1 Organización del programa 5
- 3.2 Mapeo de hilos a píxeles 6
- 3.3 Misma fórmula en CPU y GPU 6
- 3.4 Liberación de recursos 6
- 3.5 Problemas del código base 6

4 Metodología de las pruebas 7
- 4.1 Equipo de pruebas 7
- 4.2 Protocolo de medición 8

5 Uso de memoria Constante 8
- 5.1 Cómo se aplicó 8
- 5.2 Diagrama 9
- 5.3 Efecto en el tiempo de ejecución 10
- 5.4 Por qué no hubo mejora 11

6 Uso de memoria Compartida 12
- 6.1 Cómo se aplicó 12
- 6.2 Diagrama 14
- 6.3 Efecto en el tiempo de ejecución 15
- 6.4 Tamaño de bloque y ocupación 16
- 6.5 Optimizaciones probadas 17

7 Resultados y análisis 19
- 7.1 Bitácora de tiempos 19
- 7.2 Análisis 20

8 Conclusiones y recomendaciones 22
- 8.1 Retos y soluciones 23

8.2 Recomendaciones 23

9 Referencias 24

Anexo 1. Catálogo de funciones 25

Anexo 2. Bitácora de pruebas 26

Anexo 3. Cómo reproducir los resultados 29

1. Introducción

Detectar rectas en una imagen es un paso básico de la visión por computadora, sirve para encontrar carriles en una carretera, bordes de documentos o, como en este proyecto, las marcas de una pista de aterrizaje. La Transformada lineal de Hough resuelve el problema con un sistema de votación en el que cada píxel de borde vota por todas las rectas que podrían pasar por él, de modo que las rectas con más votos son las que existen en la imagen, una idea que propuso Hough (1962) y que Duda y Hart (1972) llevaron a la forma que se usa hoy.

El algoritmo es muy paralelo porque cada píxel vota de forma independiente. La imagen de prueba, `runway.pgm`, mide 800 por 600 píxeles, de los cuales 138 406 están encendidos, y como cada uno vota en 90 ángulos el total es de 12.46 millones de votos. Una GPU puede asignar un hilo a cada píxel, pero todos los votos caen en un acumulador de solo 9 000 casillas, así que muchos hilos escriben al mismo tiempo en la misma dirección y esa contención es el reto central de la implementación.

El objetivo del proyecto es implementar la transformada en CUDA y medir el efecto de tres tipos de memoria de la GPU.

1. Global, la versión base, con la imagen, las tablas trigonométricas y el acumulador en la memoria principal de la GPU.
2. Global y Constante, donde las tablas de seno y coseno pasan a la memoria constante.
3. Global, Constante y Compartida, donde cada bloque de hilos acumula sus votos en memoria compartida y al final los suma al acumulador global.

Las tres versiones se validan contra una implementación secuencial en CPU y producen exactamente el mismo acumulador. Los tiempos se midieron con eventos de CUDA en una NVIDIA GeForce RTX 5070 Ti, con 30 corridas por versión. Para explicar los resultados se diseñaron además tres experimentos, uno que compara siete formas de obtener el seno y el coseno y responde la pregunta 7a, un barrido del tamaño de bloque y una prueba de tres optimizaciones adicionales sobre la versión compartida. Todo el código y las bitácoras están en el repositorio del proyecto, `https://github.com/cru23110/proyecto3-cuda-hough`.

2. Antecedentes de la Transformada de Hough

2.1. La Transformada lineal de Hough

Una recta se puede escribir en la forma pendiente e intercepto, $y = mx + b$, pero esa forma no sirve para rectas verticales porque la pendiente se vuelve infinita. Duda y Hart (1972) propusieron describir la recta con su forma normal, es decir, con la distancia r desde el origen hasta la recta y el

ángulo θ de la perpendicular. Partiendo de

$$y = -\frac{\cos\theta}{\sin\theta}\,x + \frac{r}{\sin\theta}, \tag{1}$$

se obtiene la ecuación que usa el algoritmo,

$$r = x\cos\theta + y\sin\theta. \tag{2}$$

Cada par de valores r y θ representa una recta. Por un píxel pasan infinitas rectas, una por cada ángulo, y en el espacio de parámetros forman una sinusoide. Cuando varios píxeles están sobre la misma recta sus sinusoides se cruzan en el mismo punto, ese punto acumula muchos votos y revela la recta.

Para que el cálculo sea finito ambos parámetros se dividen en intervalos.

- El ángulo θ va de 0 a 178 grados en pasos de 2 grados, lo que da 90 ángulos, el valor de `degreeBins`.
- La distancia r va de $-r_{\text{máx}}$ a $r_{\text{máx}}$ y se divide en 100 intervalos, el valor de `rBins`. Como el origen está en el centro de la imagen, $r_{\text{máx}}$ es la mitad de la diagonal, que para una imagen de 800 por 600 son 500 píxeles.

El acumulador es entonces una matriz de 100 por 90 contadores, 9 000 casillas en total. Para cada píxel encendido y cada ángulo se calcula r con la ecuación 2, se le suma $r_{\text{máx}}$, se divide entre el ancho de un intervalo para obtener el índice `rIdx` y se incrementa la casilla `acc[rIdx * degreeBins + tIdx]`. La figura 1 muestra la imagen de entrada con sus bordes de Sobel, el acumulador resultante, conocido como espacio de Hough, y las rectas detectadas por la versión final.

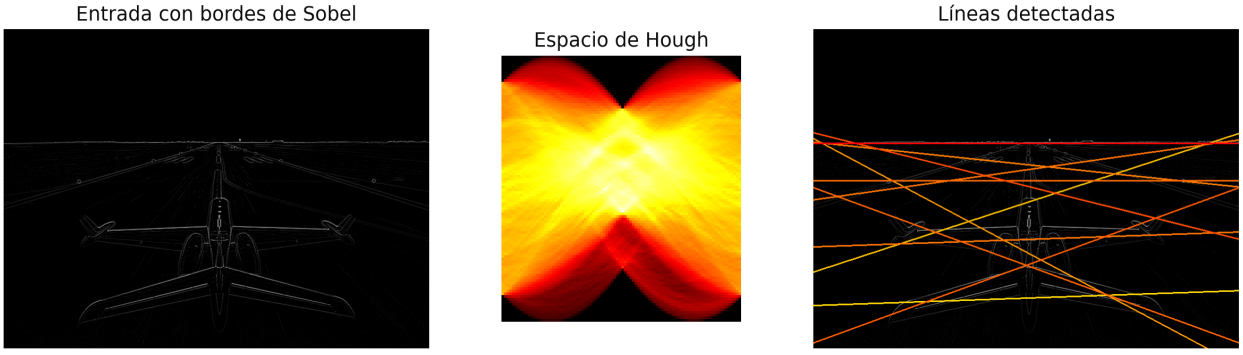


Figura 1. Etapas de la transformada

2.2. Coordenadas centradas

La imagen se guarda fila por fila en un arreglo de una dimensión y cada hilo recibe un índice `gloID`. La columna es `gloID % w` y la fila es `gloID / w`, pero el algoritmo necesita el origen en el centro de la imagen, por lo que el kernel calcula dos coordenadas.

- `xCoord = gloID % w - xCent` es la columna medida desde el centro, negativa a la izquierda.
- `yCoord = yCent - gloID / w` invierte el eje vertical, porque en la imagen la fila 0 está arriba y las filas crecen hacia abajo, mientras que en el plano cartesiano y crece hacia arriba.

Con el origen en el centro, r queda entre $-r_{\text{máx}}$ y $r_{\text{máx}}$. Si el origen estuviera en una esquina, r podría llegar al doble de esa distancia y con las mismas 100 casillas cada una cubriría el doble de píxeles, con lo que se perdería precisión. La figura 2 muestra el mapeo para el hilo con `gloID = 1000`.

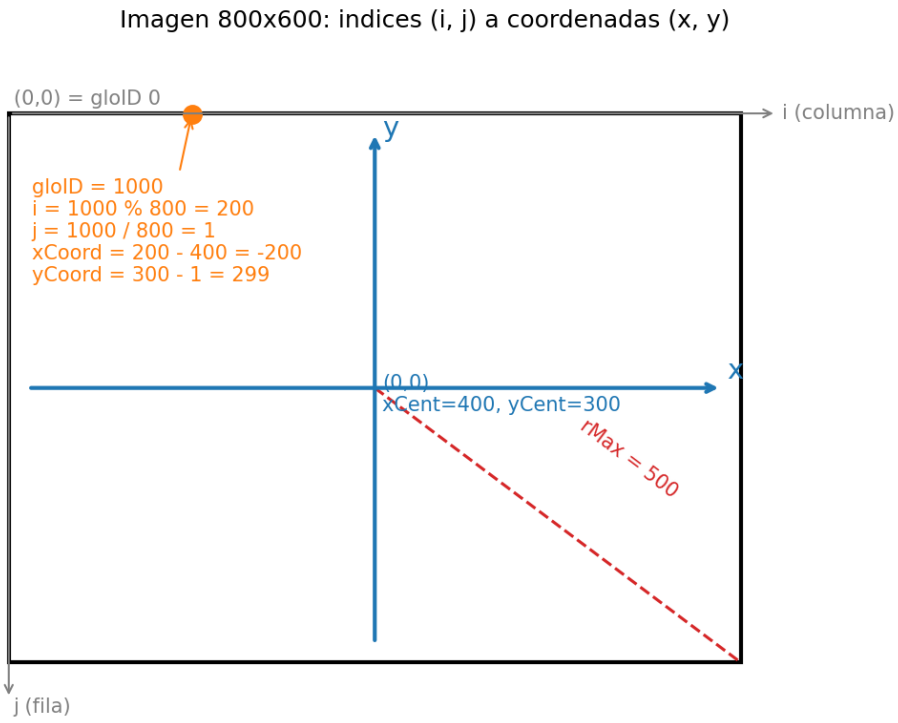


Figura 2. Mapeo a coordenadas centradas

2.3. Selección de rectas

No todas las casillas con votos son rectas. Se usa el umbral estadístico que sugiere el enunciado, el promedio más dos desviaciones estándar de las casillas con votos, que en `runway.pgm` equivale a 3 594.6 votos. Una recta real reparte votos en las casillas vecinas, así que el umbral por sí solo

produce 229 rectas casi paralelas. Para resolverlo se aplica una supresión de no-máximos, en la que una casilla cuenta como recta solo si además es el máximo de su vecindad de 5 por 5. La vecindad es circular en el ángulo, porque la recta de 178 grados es vecina de la recta de 0 grados con r de signo opuesto. Con ese filtro quedan 11 rectas, como muestra la figura 3, donde el color va de amarillo para las rectas con menos votos a rojo para las de más votos.

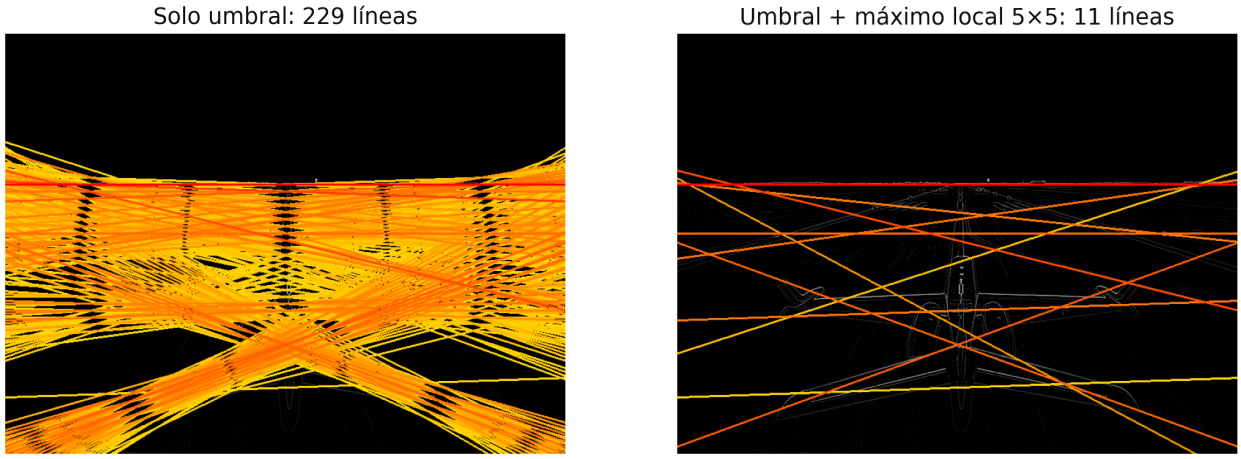


Figura 3. Efecto de suprimir no-máximos

2.4. Jerarquía de memoria de la GPU

Una GPU de NVIDIA ejecuta los hilos en grupos de 32, llamados warps, dentro de multiprocesadores que se abrevian SM. Los hilos se organizan en bloques y cada bloque corre completo en un solo SM. Las tres memorias que usa este proyecto difieren en tamaño, alcance y latencia, como describen NVIDIA (2025) y Hwu et al. (2022), y como resume la tabla 1.

Memoria	Tamaño	Alcance	Uso en este proyecto
Global	16 GB en la RTX 5070 Ti	Todos los hilos y la CPU	Imagen `pic` y acumulador `acc`, con alta latencia y sumas atómicas que se hacen en fila si apuntan a la misma dirección
Constante	64 KB, con caché por SM	Todos los hilos, solo lectura en el kernel	Tablas `d_Cos` y `d_Sin`, 720 bytes, un valor se difunde a todo el warp
Compartida	Hasta 48 KB por bloque	Hilos del mismo bloque	Acumulador local `localAcc` de 36 000 bytes, dentro del chip y de baja latencia

Tabla 1. Memorias de la GPU usadas

La guía de programación de CUDA resume así la ventaja de la memoria compartida, «como está dentro del chip, la memoria compartida tiene un ancho de banda mucho mayor y una latencia

mucho menor que la memoria local o la global»[1] (NVIDIA, 2025). El acumulador debe actualizarse con funciones atómicas, que según la misma guía realizan «una operación atómica de lectura, modificación y escritura sobre una palabra de 32, 64 o 128 bits que reside en memoria global o compartida»[2] (NVIDIA, 2025). Sin ellas, dos hilos que incrementan la misma casilla al mismo tiempo pueden leer el mismo valor y perder un voto.

3. Descripción de los programas

3.1. Organización del programa

El proyecto tiene tres programas, uno por versión, llamados `hough_global`, `hough_const` y `hough_shared`. Los tres parten del código base del curso, que sigue el ejemplo de la Transformada de Hough del libro de Barlas (2022), y comparten el código de la carpeta `common`, que contiene la lectura de imágenes PGM, las constantes del algoritmo, la versión en CPU, la validación, el reporte de tiempos, la detección y el dibujo de rectas y la escritura de imágenes PNG. De esta forma la única diferencia entre las versiones es el kernel y la manera de cargar las tablas trigonométricas, y la comparación de tiempos es justa. Un `Makefile` compila todo con `nvcc -O3 -arch=native`.

El código usa un estilo de documentación propio y uniforme, con comentarios de pocas palabras que empiezan con una etiqueta entre corchetes que indica el tema. Las etiquetas son las mismas en las tres versiones, `[ID]` para los índices del hilo, `[COORD]` para las coordenadas centradas, `[ATOMIC]` para las sumas atómicas, `[CONST]` y `[SHARED]` para cada memoria, `[BARRERA 1]` y `[BARRERA 2]` para las barreras, `[VOLCADO]` para la suma final, y en el programa principal `[HOST]`, `[GPU]`, `[TIEMPO]`, `[SALIDA]` y `[LIMPIEZA]`. Así se puede buscar una etiqueta y encontrar la misma etapa en cada versión.

Cada programa sigue los mismos pasos.

1. Leer la imagen y calcular $r_{\text{máx}}$ y el ancho de cada intervalo de r.
2. Calcular en la CPU las tablas `pcCos` y `pcSin` y el acumulador de referencia.
3. Copiar la imagen y las tablas a la GPU.
4. Ejecutar el kernel una vez de calentamiento y luego las veces que se van a medir con eventos de CUDA.
5. Copiar el acumulador a la CPU, validarlo casilla por casilla y generar las imágenes de salida.
6. Liberar toda la memoria y reiniciar el dispositivo.

[1] Traducción propia de NVIDIA Corporation, 2025, *CUDA C++ Programming Guide*, sección Shared Memory.
[2] Traducción propia de NVIDIA Corporation, 2025, *CUDA C++ Programming Guide*, sección Atomic Functions.

3.2. Mapeo de hilos a píxeles

Se lanza un hilo por píxel en una malla de una dimensión, con los bloques necesarios para cubrir la imagen y 256 hilos por bloque como valor por defecto, que se puede cambiar con la opción `--block`. El índice global del hilo es

```
int gloID = blockIdx.x * blockDim.x + threadIdx.x;
```

Los últimos hilos del último bloque pueden quedar fuera de la imagen, por eso todo el trabajo se protege con la condición `gloID < w * h`, y los píxeles apagados no votan.

3.3. Misma fórmula en CPU y GPU

Al principio la versión en GPU difería de la versión en CPU en algunas casillas y la causa era el redondeo. El compilador `nvcc` fusiona la multiplicación y la suma de `x*cos + y*sin` en una sola instrucción, mientras que el compilador de la CPU hacía dos operaciones separadas, de modo que un voto en la frontera entre dos casillas podía caer de un lado en la CPU y del otro en la GPU. La solución fue escribir una sola función `rIndex`, marcada con `__host__ __device__`, que usa `fmaf` de forma explícita para que ambos lados hagan exactamente las mismas operaciones. La función también limita el índice a `rBins - 1`, porque el píxel de una esquina puede producir un valor de r igual a $r_{\text{máx}}$ y salirse del arreglo. Con eso las tres versiones tienen cero diferencias contra la CPU en los 12 456 540 votos.

3.4. Liberación de recursos

Al final cada programa destruye los eventos, libera con `cudaFree` la imagen y el acumulador, y también las tablas en la versión global, libera con `delete[]` los arreglos de la CPU y llama a `cudaDeviceReset()`. La memoria constante no se libera porque es estática y existe mientras el módulo esté cargado. Todas las llamadas a CUDA pasan por la macro `CUDA_CHECK`, que imprime el archivo, la línea y el error si algo falla.

3.5. Problemas del código base

La tabla 2 resume los problemas que se encontraron en el código base y cómo se resolvió cada uno.

Problema	Solución
No compilaba, faltaba la clase `PGMImage` y el `Makefile` no servía	Se implementó el lector PGM y un `Makefile` con una regla para las tres versiones
Faltaba el cálculo de `gloID` y la liberación de memoria	Se completaron, como piden los incisos 1a y 1b, y se agregó `cudaDeviceReset()`
Diferencias entre CPU y GPU por el redondeo	Función común `rIndex` con `fmaf`
`rIdx` podía valer 100 en la esquina	Se limita a `rBins - 1`
El umbral dibujaba 229 rectas	Supresión de no-máximos de 5 por 5, circular en el ángulo, que deja 11 rectas
El `return` temprano no se puede usar con `__syncthreads()`	Se reemplazó por un `if`, como explica la sección 6

Tabla 2. Problemas del código base

4. Metodología de las pruebas

Todas las pruebas usan la misma imagen, `runway.pgm`, y el mismo equipo, para que las diferencias de tiempo se deban solo al uso de las memorias. Para comparar las versiones se usa la aceleración, que indica cuántas veces más rápida es una versión respecto de otra y se calcula dividiendo sus tiempos, de modo que una aceleración de 4 significa que la versión tarda la cuarta parte. Como resumen de cada grupo de corridas se usa la mediana, que no se mueve con las corridas aisladas que salen mucho más lentas, y siempre se reporta también el mínimo, que es el mejor tiempo que logró el kernel.

4.1. Equipo de pruebas

La tabla 3 describe el equipo y los parámetros que se usaron en todas las mediciones.

GPU	NVIDIA GeForce RTX 5070 Ti, 70 SM y 16 GB
Controlador y CUDA	616.56 y CUDA 12.8, con `nvcc -O3 -arch=native`
CPU	AMD Ryzen 7 7800X3D
Sistema	Windows 11 con Docker, imagen `nvidia/cuda:12.8.1-devel`
Entrada	`runway.pgm`, 800 por 600, 138 406 píxeles encendidos
Parámetros	90 ángulos, 100 intervalos de r, bloques de 256 hilos

Tabla 3. Entorno de pruebas

4.2. Protocolo de medición

Siguiendo a Harris (2012), el kernel se mide con dos eventos de CUDA que se registran antes y después de la llamada, y `cudaEventSynchronize` espera a que el kernel termine. El protocolo de medición tiene cuatro reglas.

- La primera ejecución se descarta porque incluye la carga del contexto y del módulo.
- El acumulador se pone en cero con `cudaMemset` fuera del intervalo medido.
- Las tres versiones se ejecutan intercaladas en 3 rondas de 10 corridas, para que la temperatura o la carga del sistema no favorezcan a ninguna.
- Cada corrida se agrega a un archivo CSV dentro de la carpeta `results`.

5. Uso de memoria Constante

5.1. Cómo se aplicó

En la versión global las tablas de coseno y seno se reservaban con `cudaMalloc`, se copiaban con `cudaMemcpy` y se pasaban como argumentos al kernel. En `src/hough_const.cu` se declararon en el encabezado del programa, fuera del `main` y con alcance global, como `__constant__ float d_Cos[degreeBins]` y `__constant__ float d_Sin[degreeBins]`, que ocupan 720 bytes de los 64 KB disponibles. La CPU calcula los 90 valores una sola vez con `buildTrigTables` y los copia a la memoria constante con `cudaMemcpyToSymbol`. Como los símbolos son globales, el kernel `GPU_HoughTranConst` ya no los recibe como parámetros y el programa ya no necesita `cudaMalloc` ni `cudaFree` para ellos, porque la memoria constante es estática, con lo que se cumplen los incisos 5a, 5b y 5c. Dentro del kernel cada hilo recorre los 90 ángulos en el mismo orden, así que en la iteración `tIdx` los 32 hilos de un warp leen la misma dirección `d_Cos[tIdx]`, que es el patrón ideal de esta memoria porque la caché de constantes del SM atiende la lectura una vez y la difunde a todo el warp. El resto del kernel no cambia, la imagen se sigue leyendo de la memoria global y los votos se siguen sumando con `atomicAdd` sobre el acumulador global. Los listados 1 y 2 muestran el kernel y la copia de las tablas.

Listado 1. Kernel con memoria constante

```
// [CONST] tablas cos/sin
__constant__ float d_Cos[degreeBins];
__constant__ float d_Sin[degreeBins];

__global__ void GPU_HoughTranConst(const unsigned char *__restrict__ pic, int w, int h,
    int *acc,
                                   float rMax, float rScale) {
  // [ID] un hilo, un píxel
  int gloID = blockIdx.x * blockDim.x + threadIdx.x;
  if (gloID >= w * h || pic[gloID] == 0) return;

  // [COORD] origen al centro
  int xCoord = gloID % w - w / 2;
  int yCoord = h / 2 - gloID / w;

  for (int tIdx = 0; tIdx < degreeBins; tIdx++) {
    // [CONST] mismo tIdx: broadcast
    int rIdx = rIndex(xCoord, yCoord, d_Cos[tIdx], d_Sin[tIdx], rMax, rScale);
    // [ATOMIC] varios píxeles, mismo bin
    atomicAdd(&acc[rIdx * degreeBins + tIdx], 1);
  }
}
```

Listado 2. Copia a memoria constante

```
  // [CONST] host a símbolo
  CUDA_CHECK(cudaMemcpyToSymbol(d_Cos, pcCos, sizeof(float) * degreeBins));
  CUDA_CHECK(cudaMemcpyToSymbol(d_Sin, pcSin, sizeof(float) * degreeBins));
```

5.2. Diagrama

La figura 4 muestra las entradas, las etapas y la salida del kernel con memoria constante.

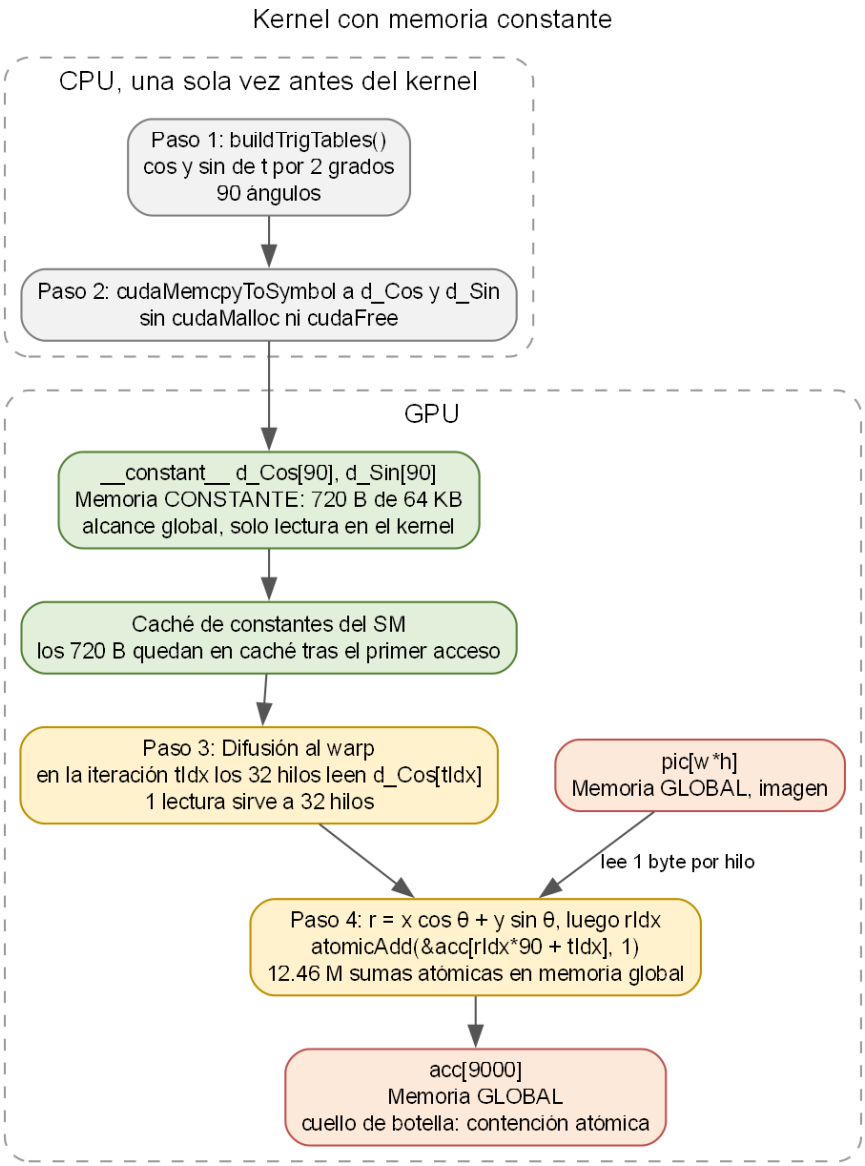


Figura 4. Uso de memoria constante

5.3. Efecto en el tiempo de ejecución

Con bloques de 256 hilos y 30 corridas, la mediana de la versión constante es de 0.507 ms, igual a la de la versión global. Las medias son distintas, 0.565 ms contra 0.653 ms, pero solo porque la versión global tuvo más corridas con picos de ruido del sistema, que se analizan en la sección 7. Una prueba t de Welch entre ambas da $p = 0.065$, por lo que la diferencia no es estadísticamente significativa y se concluye que la memoria constante no mejoró el tiempo.

5.4. Por qué no hubo mejora

Para responder la pregunta 7a con datos y no solo con teoría se escribió el experimento `variants.cu`, en la carpeta `src/experiments`. El experimento ejecuta siete kernels idénticos salvo por la forma de obtener el coseno y el seno, todos con el acumulador en memoria global, intercalados y con 30 corridas cada uno. La tabla 4 y la figura 5 muestran los tiempos en milisegundos. En las variantes cuyo nombre termina en `_div` cada hilo del warp recorre los ángulos con un desfase distinto, y la última columna cuenta las casillas que difieren de la CPU.

Variante	Mediana	Mínimo	Diferencias
cosf_kernel	0.510	0.502	2
sincosf_intrin	0.509	0.500	65
global	0.502	0.497	0
global_ldg	0.505	0.495	0
constant	0.510	0.499	0
constant_div	0.327	0.321	0
global_ldg_div	0.321	0.316	0

Tabla 4. Variantes de acceso trigonométrico

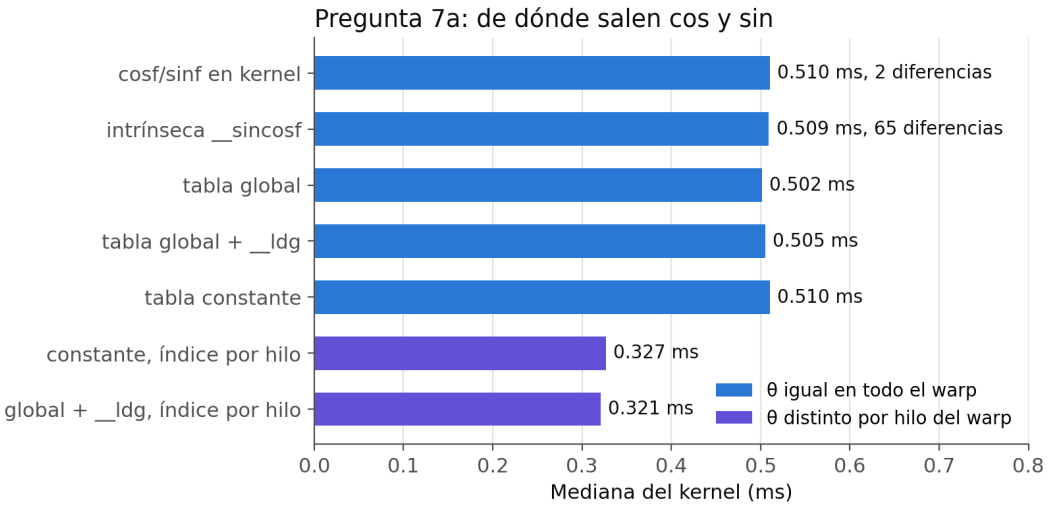


Figura 5. Tiempo según origen trigonométrico

Los resultados muestran cuatro cosas.

1. Leer el seno y el coseno no es el cuello de botella. Las cinco variantes con acceso uniforme tardan lo mismo, entre 0.502 y 0.510 ms, ya sea con una tabla en memoria constante, una tabla en memoria global o calculando `cosf` en el kernel. Una tabla de 720 bytes queda en la caché

del SM después del primer acceso, así que la versión global ya leía las tablas desde caché, un efecto que el enunciado anticipa cuando dice que desde la arquitectura Fermi la caché L2 «tiene menos latencia y satisface muchas de las necesidades de transacciones de memoria» (Universidad del Valle de Guatemala, 2024).

2. El cuello de botella son las sumas atómicas sobre el acumulador global. Cada píxel hace 90 votos, lo que da 12.46 millones de operaciones atómicas sobre solo 9 000 direcciones. En las variantes con desfase los 32 hilos de un warp votan en 32 columnas de ángulo diferentes en lugar de la misma, el trabajo es idéntico y el resultado sigue siendo exacto, pero el tiempo baja de 0.50 a 0.32 ms, un 36 % menos, y lo único que cambió fue la contención.

3. La memoria constante se vuelve lenta con accesos divergentes. La guía de CUDA explica que en la memoria constante «la petición se divide en tantas peticiones separadas como direcciones de memoria distintas haya en la petición inicial, lo que reduce el rendimiento en un factor igual al número de peticiones separadas»[3] (NVIDIA, 2025). En este experimento `constant_div` es apenas más lenta que `global_ldg_div`, 0.327 contra 0.321 ms, porque las sumas atómicas dominan el tiempo. Dentro de la versión compartida, donde ya no dominan, el mismo patrón hace a la memoria constante 2.4 veces más lenta que la memoria global, como se mide en la sección 6.5. La memoria constante solo conviene cuando todo el warp lee la misma dirección, que es justo el patrón del kernel final.

4. La tabla precalculada también es necesaria para obtener el resultado exacto. Calcular `cosf` en la GPU produce 2 casillas distintas de la CPU y la función intrínseca `__sincosf`, que es menos precisa, produce 65, mientras que con la tabla la CPU y la GPU usan exactamente los mismos valores.

En resumen, la memoria constante se aplicó correctamente y es la memoria adecuada para estas tablas, pero no podía mejorar el tiempo porque la lectura de las tablas nunca fue lo que limitaba el kernel. Para acelerarlo había que reducir la contención atómica, que es lo que hace la memoria compartida.

6. Uso de memoria Compartida

6.1. Cómo se aplicó

El enunciado explica por qué el tamaño del acumulador está limitado en CUDA, «en el caso de CUDA en realidad está restringido por el tamaño de la memoria Compartida, ya que para beneficiarnos de la velocidad de acceso a esta memoria, mantenemos un acumulador local por cada

[3] Traducción propia de NVIDIA Corporation, 2025, *CUDA C++ Programming Guide*, sección Constant Memory.

bloque de hilos» (Universidad del Valle de Guatemala, 2024). En las versiones anteriores los 12.46 millones de votos se hacían con `atomicAdd` directamente sobre el acumulador global, de solo 9 000 direcciones, y la sección 5 mostró que esa contención es lo que limita el kernel. La versión final, `src/hough_shared.cu`, usa las tres memorias, con la imagen en global, las tablas en constante y un acumulador local por bloque llamado `localAcc`, de 9 000 enteros que ocupan 36 000 bytes, declarado con `__shared__` como piden los incisos 8a y 8b. Un `static_assert` verifica al compilar que cabe en los 48 KB disponibles por bloque. El kernel, que aparece en el listado 3, trabaja en tres etapas separadas por dos barreras `__syncthreads()`.

1. En la inicialización del inciso 8c cada hilo usa su índice local `locID` para poner en cero las casillas `locID`, `locID + blockDim.x` y así sucesivamente hasta 9 000, de modo que los 256 hilos se reparten el trabajo y cada uno limpia unas 35 casillas.

2. Después de la primera barrera, que corresponde al inciso 8d, cada hilo procesa su píxel y vota con `atomicAdd` sobre `localAcc`, como pide el inciso 8e. La operación sigue siendo atómica porque los hilos del bloque comparten el acumulador, pero ahora ocurre en memoria dentro del chip y solo compiten los 256 hilos del bloque.

3. Después de la segunda barrera, la del inciso 8f, un ciclo que empieza en `locID` recorre `localAcc` con el mismo salto y suma cada casilla al acumulador global con un solo `atomicAdd`, omitiendo las casillas en cero, con lo que se cumple el inciso 8g.

La barrera 1 evita que un hilo vote en una casilla que otro hilo todavía no ha puesto en cero, y la barrera 2 evita que un hilo copie una casilla a la memoria global mientras otros hilos del bloque siguen votando en ella. Por eso el kernel no puede usar el `return` temprano de las versiones anteriores para los hilos sobrantes o los píxeles apagados. La guía de CUDA advierte que «__syncthreads() se permite en código condicional solo si la condición se evalúa de forma idéntica en todo el bloque de hilos, de lo contrario es probable que la ejecución se cuelgue o produzca efectos no deseados»[4] (NVIDIA, 2025). La votación se protege entonces con un `if`, y todos los hilos del bloque llegan a las dos barreras y participan en la inicialización y el volcado.

[4]Traducción propia de NVIDIA Corporation, 2025, *CUDA C++ Programming Guide*, sección Synchronization Functions.

Listado 3. Kernel con memoria compartida

```
__global__ void GPU_HoughTranShared(const unsigned char *__restrict__ pic, int w, int h,
    int *acc,
                                    float rMax, float rScale) {
  // [SHARED] acumulador del bloque
  __shared__ int localAcc[accSize];
  int locID = threadIdx.x;
  // [ID] local y global
  int gloID = blockIdx.x * blockDim.x + locID;

  for (int i = locID; i < accSize; i += blockDim.x) localAcc[i] = 0;
  // [BARRERA 1] init terminada
  __syncthreads();

  // [BARRERA] sin return temprano
  if (gloID < w * h && pic[gloID] > 0) {
    int xCoord = gloID % w - w / 2;
    int yCoord = h / 2 - gloID / w;
    for (int tIdx = 0; tIdx < degreeBins; tIdx++) {
      int rIdx = rIndex(xCoord, yCoord, d_Cos[tIdx], d_Sin[tIdx], rMax, rScale);
      // [ATOMIC] voto en shared
      atomicAdd(&localAcc[rIdx * degreeBins + tIdx], 1);
    }
  }
  // [BARRERA 2] votos terminados
  __syncthreads();

  // [VOLCADO] solo bins con votos
  for (int i = locID; i < accSize; i += blockDim.x)
    if (localAcc[i] > 0) atomicAdd(&acc[i], localAcc[i]);
}
```

6.2. Diagrama

La figura 6 muestra el recorrido de un bloque, con sus cuatro etapas y las dos barreras.

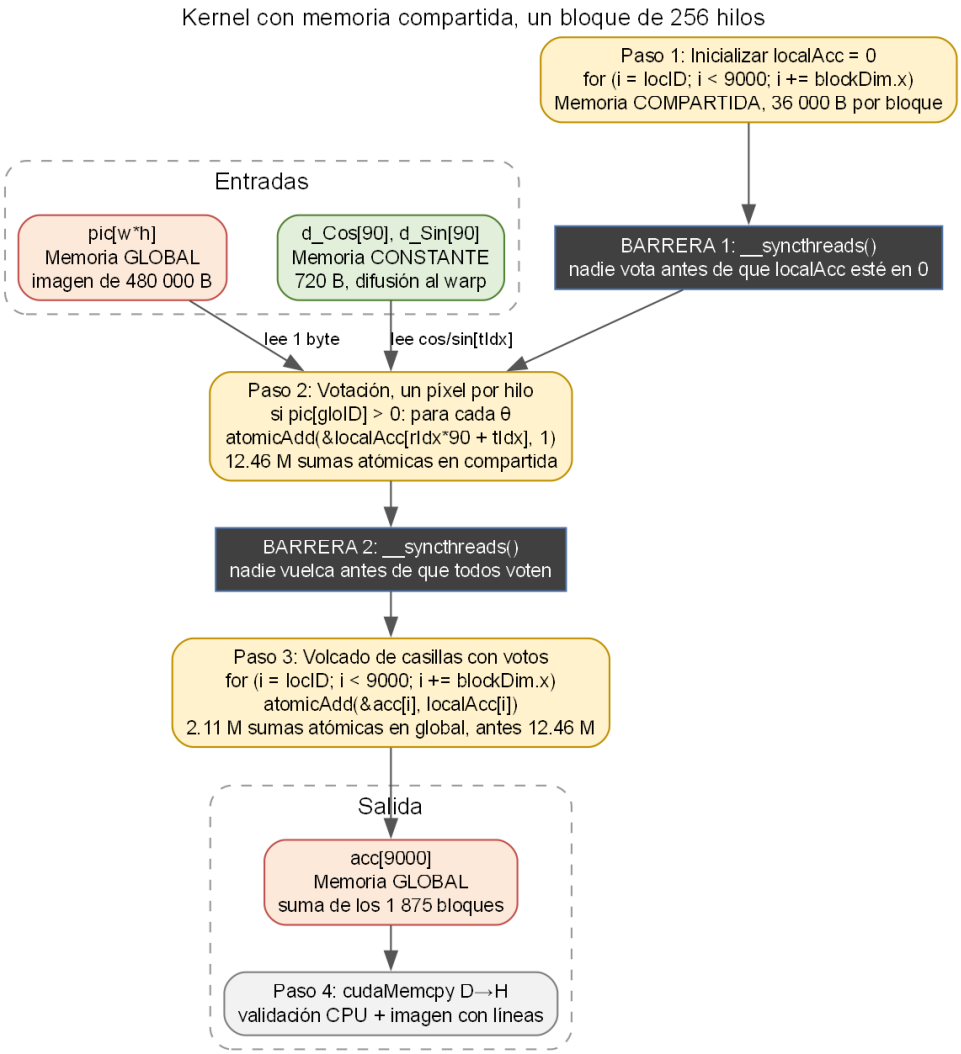


Figura 6. Uso de memoria compartida

6.3. Efecto en el tiempo de ejecución

Con bloques de 256 hilos y 30 corridas, la mediana de la versión compartida es de 0.113 ms contra 0.507 ms de la versión global, es decir, 4.49 veces más rápida, una diferencia significativa según la prueba de Welch. Sus 30 corridas están entre 0.1129 y 0.1132 ms, mientras que la versión global varió entre 0.497 y 1.091 ms por el ruido de medición que se analiza en la sección 7. La mejora tiene dos causas.

1. Hay muchas menos sumas atómicas en memoria global. Los 12.46 millones de votos se hacen en memoria compartida y a la memoria global solo llegan las sumas del volcado, unas 2.11 millones, 5.9 veces menos. La cifra se calculó contando las combinaciones distintas de bloque y casilla que produce la imagen.

2. Hay menos contención. En lugar de que los 480 000 hilos compitan por un único acumulador, cada uno de los 1 875 bloques compite solo dentro de su propio acumulador local, que además tiene mucha menor latencia.

El experimento `variants.cu` también midió dos detalles del volcado con bloques de 256 hilos, que aparecen en la tabla 5 en milisegundos. Omitir las casillas en cero ahorra 3.5 % del tiempo, porque cada bloque toca en promedio solo 1 128 de las 9 000 casillas. Además, la variante `solo_init_volcado`, que inicializa y vuelca sin votar y por eso no coincide con la CPU, tarda 0.023 ms, lo que indica que alrededor del 20 % del tiempo total es un costo fijo por bloque que no depende de cuántos píxeles tenga.

Variante	Mediana	Mínimo	Diferencias
skip_ceros	0.113	0.113	0
volcar_todo	0.117	0.117	0
solo_init_volcado	0.023	0.023	8068

Tabla 5. Variantes del volcado

6.4. Tamaño de bloque y ocupación

La tabla 6 y la figura 7 muestran el barrido de tamaño de bloque. Cada valor es el mínimo de 30 corridas en milisegundos, porque el mínimo es el estadístico que no se ve afectado por el ruido de medición, y las dos últimas columnas indican cuántos bloques caben a la vez en un SM. Con bloques más grandes hay menos bloques, con 512 hilos son 938 en lugar de 1 875, y el trabajo de inicializar y volcar baja de 33.8 a 16.9 millones de operaciones. Por otro lado, los 36 KB de memoria compartida por bloque limitan la ocupación, ya que solo caben 2 bloques por SM y con 256 hilos quedan activos 512 de los 1 536 hilos posibles, apenas un 33 %. Con 512 hilos por bloque se duplican los hilos activos y se reduce a la mitad el costo fijo, por lo que el kernel baja a 0.074 ms, 6.8 veces más rápido que la versión global. El experimento de la sección 6.5 separa los dos efectos, de los 0.039 ms que se ahorran al pasar de 256 a 512 hilos solo 0.012 ms, un 32 %, vienen del costo fijo, y el 68 % restante viene de que la votación es más rápida con el doble de hilos activos por SM. Con 1 024 hilos solo cabe un bloque por SM y el tiempo deja de mejorar. Las versiones global y constante no usan memoria compartida, su tiempo casi no cambia porque las limita la contención atómica y solo sube un poco con 1 024 hilos, cuando su ocupación baja a 66 %. Se mantuvo 256 como valor por defecto para comparar las tres versiones en igualdad de condiciones y el mejor tamaño, 512, se reporta aparte.

Hilos por bloque	Global	Constante	Compartida	Bloques por SM, global	Bloques por SM, compartida
128	0.497	0.498	0.213	12	2
256	0.496	0.497	0.113	6	2
512	0.499	0.503	0.074	3	2
1024	0.517	0.519	0.076	1	1

Tabla 6. Barrido de tamaño de bloque

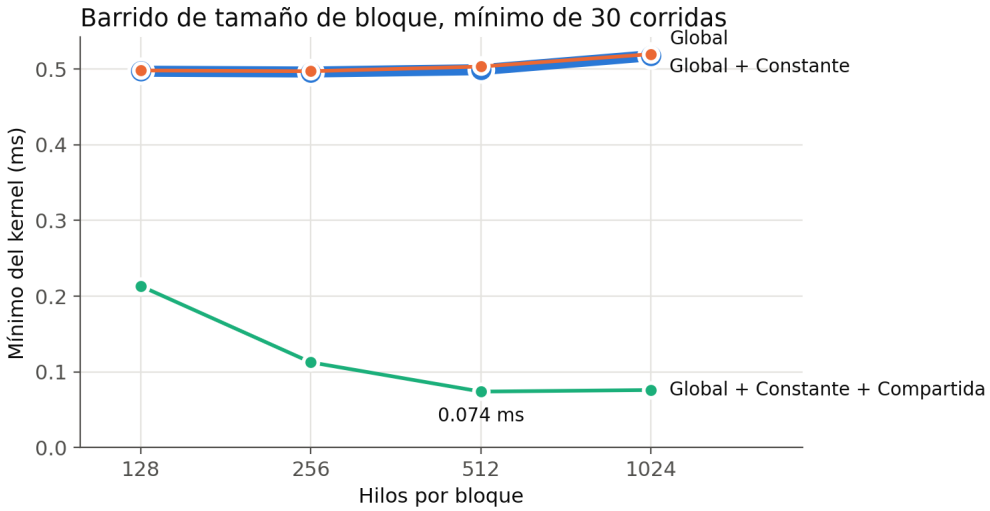


Figura 7. Tiempo según tamaño de bloque

6.5. Optimizaciones probadas

Para saber si la versión compartida se podía mejorar más, el experimento `optimizaciones.cu` probó tres ideas sobre el mismo kernel, con 30 corridas intercaladas y validación contra la CPU en cada variante. La tabla 7 y la figura 8 muestran los tiempos en milisegundos. Las tres últimas filas de la tabla no votan, solo inicializan y vuelcan para medir el costo fijo de cada configuración, y por eso no coinciden con la CPU.

Variante	Bloques	Mediana	Mínimo	Diferencias
Versión final, 256 hilos	1875	0.113	0.111	0
Versión final, 512 hilos	938	0.074	0.072	0
Tabla global con `__ldg`, 512	938	0.072	0.070	0
θ por hilo, tabla constante	938	0.329	0.325	0
θ por hilo, tabla global `__ldg`	938	0.136	0.135	0
Varios píxeles por hilo, 256	140	0.160	0.160	0
Varios píxeles por hilo, 512	140	0.111	0.111	0
Solo inicializar y volcar, 256	1875	0.023	0.023	8068
Solo inicializar y volcar, 512	938	0.011	0.011	8068
Solo inicializar y volcar, malla fija	140	0.004	0.004	8068

Tabla 7. Optimizaciones de la versión compartida

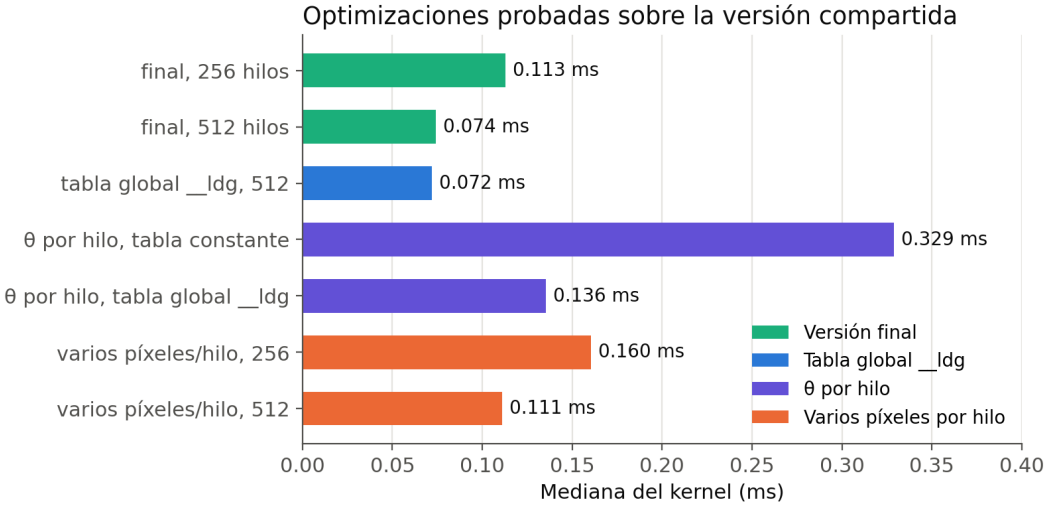


Figura 8. Tiempo de cada optimización

1. La primera idea fue leer las tablas de memoria global con `__ldg`, que usa la caché de solo lectura. Con el acceso uniforme del kernel final es apenas 3 % más rápida que la memoria constante, 0.072 contra 0.074 ms, lo que confirma lo visto en la pregunta 7a, la memoria constante no aporta velocidad en esta GPU aunque tampoco resta.

2. La segunda idea fue desfasar el ángulo por hilo. En la versión global ese desfase redujo la contención y bajó el tiempo 36 %, pero dentro de la versión compartida pasa lo contrario y con las tablas en memoria global el kernel sube de 0.072 a 0.136 ms, porque las sumas atómicas en memoria compartida ya no son el cuello de botella y el desfase solo agrega trabajo. Con las tablas en memoria constante el efecto es mucho peor, 0.329 ms, 2.4 veces más lento que la misma variante con `__ldg`, ya que cada hilo del warp lee una dirección distinta de `d_Cos` y la memoria constante divide cada lectura en hasta 32 peticiones. Es la medición más clara del proyecto sobre cuándo la memoria constante es una mala elección.

3. La tercera idea fue procesar varios píxeles por hilo. En lugar de un hilo por píxel se lanzan solo los 140 bloques que caben a la vez en la GPU y cada hilo recorre unos 7 píxeles con un salto fijo. El costo fijo casi desaparece, 0.004 ms contra 0.011 ms, pero el kernel tarda 0.111 ms, 1.5 veces más que la versión final. La causa es el desbalance de carga, porque con el reparto fijo a cada bloque le toca siempre la misma franja de columnas y los píxeles encendidos se concentran en el centro de la imagen, de modo que el bloque más cargado recibe 1.46 veces más píxeles encendidos que el promedio y el kernel espera al más lento. Con 938 bloques pequeños el planificador de la GPU reparte la carga dinámicamente y ese desbalance se compensa.

En resumen, la versión final con 512 hilos por bloque ya está cerca del mejor diseño probado. Las dos optimizaciones que parecían prometedoras empeoran el tiempo y medirlas evitó recomendarlas sin evidencia.

El acumulador de la versión compartida es idéntico casilla por casilla al de la CPU, con cero diferencias en los 12 456 540 votos, y también al de las otras dos versiones. El programa `scripts/compare_outputs.py` confirma que las tres versiones generan el mismo acumulador y la misma imagen de salida, como se muestra en el anexo 2.

7. Resultados y análisis

7.1. Bitácora de tiempos

La tabla 8 muestra las primeras 10 mediciones del kernel de cada versión, en milisegundos y tomadas con eventos de CUDA. Las 30 corridas completas están en el anexo 2 y en los archivos CSV de la carpeta `results`. La tabla 9 resume las 30 corridas, también en milisegundos, con la aceleración calculada sobre la mediana, y la figura 9 muestra a la izquierda cada corrida y a la derecha la aceleración respecto de la versión global.

Corrida	Global	Global + Constante	Global + Constante + Compartida
1	0.503	0.504	0.113
2	0.510	0.504	0.113
3	0.501	0.504	0.113
4	1.081	0.503	0.113
5	0.848	0.505	0.113
6	0.827	0.507	0.113
7	0.506	0.508	0.113
8	0.497	0.508	0.113
9	0.820	0.506	0.113
10	0.507	0.505	0.113

Tabla 8. Bitácora de diez corridas

Versión	Media	Desviación	Mediana	Mínimo	Máximo	IC 95 %	Aceleración
Global	0.653	0.223	0.507	0.497	1.091	±0.083	1.00×
Global + Constante	0.565	0.123	0.507	0.500	0.852	±0.046	1.00×
Global + Constante + Compartida	0.113	0.000	0.113	0.113	0.113	±0.000	4.49×

Tabla 9. Resumen de treinta corridas

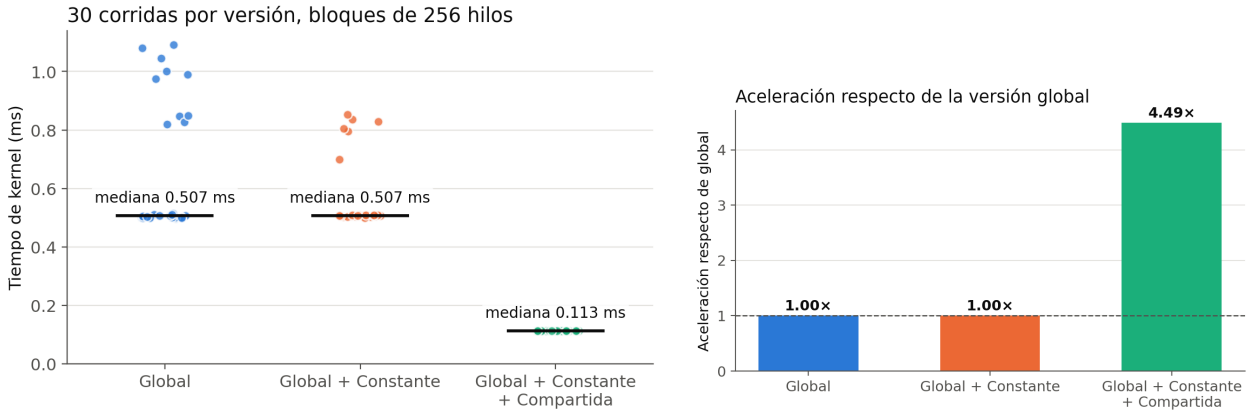


Figura 9. Tiempo del kernel por versión

7.2. Análisis

Las versiones global y constante tienen corridas aisladas de 0.7 a 1.1 ms aunque su valor típico es de 0.50 ms. Para saber si esos picos vienen del kernel o del sistema se hizo un experimento de control con 200 corridas de la versión global y 200 de la versión compartida con bloques de 64 y de 32 hilos, que la vuelven tan lenta o más que la global. La tabla 10 muestra los tiempos en

milisegundos y el porcentaje de corridas lentas, que son las que superan en más de 20 % al mínimo. Los kernels compartidos largos muestran los mismos saltos, en escalones de unos 0.15 ms que se suman a un tiempo base fijo, mientras que ningún kernel de menos de 0.25 ms tuvo un solo salto en 210 corridas. El retraso depende entonces de cuánto dura el kernel y no de la memoria que usa, así que es interferencia externa, consistente con que la GPU comparte su tiempo con el escritorio de Windows. Por eso el tiempo base de cada kernel es su mínimo, que se mantuvo entre 0.496 y 0.497 ms para la versión global en las cuatro sesiones de medición. La bitácora se resume con la mediana, que en estas 30 corridas queda cerca del mínimo, y siempre se reporta el mínimo junto a ella. Con los mínimos la versión compartida es 4.40 veces más rápida que la global y con las medianas 4.49 veces.

Kernel	Corridas	Mínimo	Mediana	Máximo	Corridas lentas
Compartida, bloques de 32	200	1.169	1.721	2.064	73 %
Global, bloques de 256	200	0.497	0.680	1.046	56 %
Compartida, bloques de 64	200	0.458	0.614	0.795	62 %

Tabla 10. Experimento de control de ruido

La tabla 11 aplica una prueba t de Welch entre cada par de versiones, con 30 corridas por versión y un nivel de significancia de 0.05. La diferencia entre la versión global y la constante no es significativa, mientras que la versión compartida es significativamente más rápida que las otras dos.

Comparación	t	p	Significativa
Global y constante	1.89	6.5×10^{-2}	No
Global y compartida	13.25	7.8×10^{-14}	Sí
Constante y compartida	20.13	1.4×10^{-18}	Sí

Tabla 11. Prueba t de Welch

La versión secuencial tarda 32.5 ms en la CPU, de modo que la versión global es 64 veces más rápida y la compartida 288 veces más rápida, contando solo el kernel y no las transferencias de memoria.

Las tres versiones producen el mismo acumulador, detectan las mismas 11 rectas y generan imágenes idénticas byte por byte. La figura 10 es la imagen de salida de la versión final, con las rectas dibujadas en color sobre la imagen de entrada, en rojo las de más votos y en amarillo las más débiles.

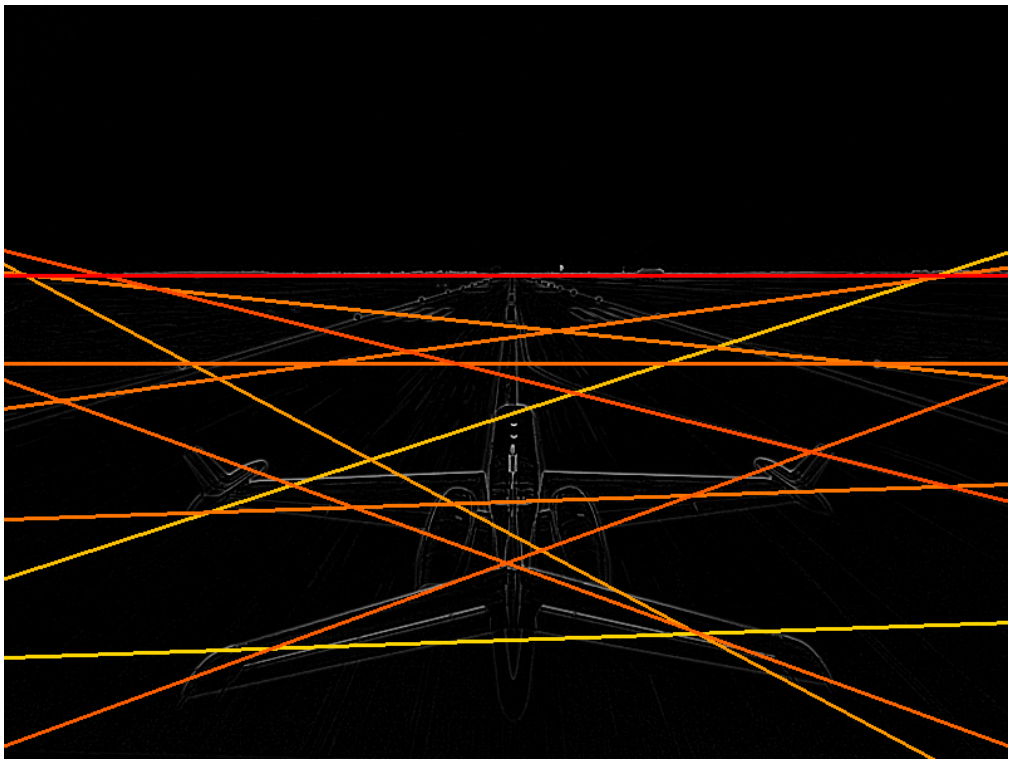


Figura 10. Rectas detectadas en la pista

8. Conclusiones y recomendaciones

1. La memoria compartida es la que acelera el algoritmo. La versión final redujo la mediana del kernel de 0.507 a 0.113 ms con bloques de 256 hilos, 4.49 veces más rápida, y a 0.074 ms con bloques de 512, porque traslada los 12.46 millones de votos a memoria dentro del chip y deja solo 2.11 millones de sumas atómicas en la memoria global, todo sin cambiar el resultado, que coincide casilla por casilla con el de la CPU.

2. La memoria constante no mejoró el tiempo, que fue de 0.507 ms en ambas versiones, porque el límite del kernel nunca fue leer las tablas de seno y coseno sino la contención de las sumas atómicas. Es la memoria adecuada cuando todo el warp lee la misma dirección, pero con accesos distintos por hilo resultó 2.4 veces más lenta que la memoria global.

3. Cada decisión de diseño tuvo que medirse para sostenerse. Las barreras obligaron a cambiar el `return` temprano por un `if`, el tamaño de bloque de 512 hilos resultó mejor sobre todo por la ocupación y no por el costo fijo, y dos optimizaciones que parecían prometedoras, desfasar el ángulo por hilo y procesar varios píxeles por hilo, empeoraron el tiempo cuando se probaron.

8.1. Retos y soluciones

La tabla 12 resume los retos de la implementación y la forma en que se resolvieron.

Reto	Solución
Pequeñas diferencias entre CPU y GPU por el redondeo de `nvcc`	Función `rIndex` compartida con `fmaf` explícito, cero diferencias
El umbral estadístico producía 229 rectas casi iguales	Supresión de no-máximos de 5 por 5, circular en el ángulo, que deja 11 rectas
La memoria constante no mostró mejora	Experimento de 7 variantes que identificó la contención atómica como causa
`return` temprano incompatible con `__syncthreads()`	Votación protegida con `if`, todos los hilos llegan a ambas barreras
El acumulador local debe caber en la memoria compartida	Ocupa 36 000 bytes de los 48 KB, verificado con `static_assert`
Costo fijo de inicializar y volcar 9 000 casillas por bloque	Recorrido con salto `blockDim.x`, omitir casillas en cero y bloques de 512
Saber si las optimizaciones conocidas ayudaban	Experimento con 7 variantes validadas, dos empeoraban el tiempo y se descartaron
Saltos aleatorios en los tiempos de los kernels largos	Experimento de control que mostró que son externos, corridas intercaladas, mínimo, mediana y prueba de Welch
Demostrar que las tres versiones dan lo mismo	Validación contra la CPU en cada ejecución y comparación de salidas

Tabla 12. Retos y soluciones

8.2. Recomendaciones

- Usar bloques de 512 hilos en la versión compartida para esta imagen y esta GPU, y repetir el barrido con `scripts/block_sweep.sh` si se cambia de equipo.
- No desfasar el ángulo dentro de la versión compartida ni combinar accesos distintos por hilo con memoria constante, porque el experimento de optimizaciones mostró que ambos empeoran el tiempo.
- Si se procesan varios píxeles por hilo, repartir el trabajo de forma dinámica, por ejemplo con un contador atómico de lotes, ya que el reparto fijo dejó un bloque con 1.46 veces la carga promedio.
- Medir en una GPU dedicada o en Linux para comprobar si desaparece la interferencia que se observó en Windows, y mover la detección de rectas a la GPU cuando se procesen muchas imágenes.

9. Referencias

Barlas, G. (2022). *Multicore and GPU programming: An integrated approach* (2.ª ed.). Morgan Kaufmann.

Duda, R. O., & Hart, P. E. (1972). Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM, 15*(1), 11–15. https://doi.org/10.1145/361237.361242

Harris, M. (2012). How to implement performance metrics in CUDA C/C++. *NVIDIA Technical Blog*. https://developer.nvidia.com/blog/how-implement-performance-metrics-cuda-cc/

Hough, P. V. C. (1962). *Method and means for recognizing complex patterns* (Patente de EE. UU. N.º 3,069,654).

Hwu, W. W., Kirk, D. B., & El Hajj, I. (2022). *Programming massively parallel processors: A hands-on approach* (4.ª ed.). Morgan Kaufmann.

NVIDIA. (2025). *CUDA C++ programming guide*, versión 12.8. Recuperado de https://docs.nvidia.com/cuda/archive/12.8.0/cuda-c-programming-guide/

Universidad del Valle de Guatemala. (2024). *Proyecto 3: CUDA* [Enunciado del curso Computación Paralela y Distribuida].

Anexo 1. Catálogo de funciones

La siguiente tabla resume las funciones principales del proyecto, con lo que reciben, lo que devuelven y para qué sirven.

Función	Entradas	Salida	Descripción
`GPU_HoughTran`	imagen, ancho, alto, acumulador, tablas en global	votos en el acumulador	Kernel con memoria global, un hilo por píxel
`GPU_HoughTranConst`	imagen, ancho, alto, acumulador	votos en el acumulador	Kernel que lee las tablas de memoria constante
`GPU_HoughTranShared`	imagen, ancho, alto, acumulador	votos en el acumulador	Kernel final con acumulador local y dos barreras
`rIndex`	coordenadas, coseno, seno, $r_{\text{máx}}$, escala	índice de r	Misma fórmula en CPU y GPU, usa `fmaf`
`CPU_HoughTran`	imagen, ancho, alto, tablas	acumulador de referencia	Versión secuencial para validar
`buildTrigTables`	ninguna	tablas de coseno y seno	Calcula los 90 valores una sola vez
`validate`	acumuladores de CPU y GPU	verdadero si coinciden	Compara casilla por casilla
`reportTimes`	versión, opciones, tiempos	estadísticas y CSV	Imprime media, desviación, mínimo y máximo y agrega las corridas al CSV
`parseArgs`	argumentos del programa	opciones o error	Lee y valida las opciones de la línea de comandos
`lineThreshold`	acumulador, factor k	umbral	Promedio más k desviaciones de las casillas con votos
`selectLines`	acumulador, umbral, NMS, top	lista de rectas	Aplica el umbral y el máximo local de 5 por 5
`drawLines`	imagen, rectas, umbral	imagen con rectas	Dibuja cada recta con un color según sus votos
`saveHoughImage`	ruta, acumulador	archivo PNG	Guarda el espacio de Hough como mapa de calor
`saveOutputs`	opciones, versión, imagen, acumulador	imágenes y archivo `.bin`	Genera todas las salidas de una versión
`printOccupancy`	kernel, hilos por bloque	bloques activos por SM	Consulta la ocupación con la API de CUDA

Función	Entradas	Salida	Descripción
`PGMImage`	ruta del archivo	imagen en memoria	Lee imágenes PGM en formato P5 o P2

Anexo 2. Bitácora de pruebas

Treinta corridas por versión

La tabla 14 contiene las 30 corridas de cada versión en milisegundos, con bloques de 256 hilos y en orden de ejecución.

Corrida	Global	Global + Constante	Global + Constante + Compartida
1	0.503	0.504	0.113
2	0.510	0.504	0.113
3	0.501	0.504	0.113
4	1.081	0.503	0.113
5	0.848	0.505	0.113
6	0.827	0.507	0.113
7	0.506	0.508	0.113
8	0.497	0.508	0.113
9	0.820	0.506	0.113
10	0.507	0.505	0.113
11	0.502	0.507	0.113
12	0.499	0.504	0.113
13	0.501	0.500	0.113
14	0.499	0.836	0.113
15	0.501	0.505	0.113
16	0.499	0.830	0.113
17	0.500	0.503	0.113
18	1.001	0.509	0.113
19	0.975	0.504	0.113
20	1.045	0.505	0.113
21	0.505	0.506	0.113
22	0.503	0.796	0.113
23	0.514	0.508	0.113
24	0.502	0.507	0.113
25	0.507	0.509	0.113
26	0.507	0.805	0.113
27	0.849	0.506	0.113
28	0.990	0.508	0.113
29	1.091	0.852	0.113
30	0.511	0.699	0.113

Tabla 14. Bitácora completa de tiempos

Comparación de las tres versiones

El programa `compare_outputs.py` compara los acumuladores y las imágenes de salida de las tres versiones, y esta es su salida.

```
Referencia: runway_global_acc.bin (12456540 votos)
runway_const_acc.bin: bins distintos = 0, imagen identica = True
runway_shared_acc.bin: bins distintos = 0, imagen identica = True
Resultado: TODAS IGUALES
```

Experimento de la pregunta 7a

Salida del experimento `variants`, con 30 corridas intercaladas por variante y bloques de 256 hilos.

```
GPU: NVIDIA GeForce RTX 5070 Ti (sm_120, 70 SMs)
grupo    variante               media_ms     min_ms   mismatch
trig     cosf_kernel              0.6541     0.5021          2
trig     sincosf_intrin           0.6679     0.5000         65
trig     global                   0.5694     0.4969          0
trig     global_ldg               0.6708     0.4951          0
trig     constant                 0.7055     0.4990          0
trig     constant_div             0.4046     0.3208          0
trig     global_ldg_div           0.4153     0.3157          0
shared   skip_ceros               0.1139     0.1128          0
shared   volcar_todo              0.1185     0.1171          0
shared   solo_init_volcado        0.0229     0.0228       8068
```

Optimizaciones de la versión compartida

Salida del experimento `optimizaciones`, con 30 corridas intercaladas por variante.

```
GPU: NVIDIA GeForce RTX 5070 Ti (sm_120, 70 SMs)
variante       bloque   grid    mediana     min_ms  mismatch
base_256          256   1875     0.1130     0.1109         0
base_512          512    938     0.0740     0.0720         0
ldg_512           512    938     0.0720     0.0700         0
div_const_512     512    938     0.3290     0.3249         0
div_ldg_512       512    938     0.1355     0.1355         0
multi_256         256    140     0.1604     0.1601         0
multi_512         512    140     0.1109     0.1109         0
fijo_256          256   1875     0.0228     0.0228      8068
fijo_512          512    938     0.0106     0.0106      8068
fijo_multi_512    512    140     0.0044     0.0044      8068
```

Ocupación por tamaño de bloque

Bloques activos por SM que reporta CUDA para cada versión y cada tamaño de bloque.

```
global,128, 12 bloques/SM, 100% de hilos
const,128, 12 bloques/SM, 100% de hilos
shared,128, 2 bloques/SM, 16% de hilos
global,256, 6 bloques/SM, 100% de hilos
const,256, 6 bloques/SM, 100% de hilos
```

```
shared,256, 2 bloques/SM, 33% de hilos
global,512, 3 bloques/SM, 100% de hilos
const,512, 3 bloques/SM, 100% de hilos
shared,512, 2 bloques/SM, 66% de hilos
global,1024, 1 bloques/SM, 66% de hilos
const,1024, 1 bloques/SM, 66% de hilos
shared,1024, 1 bloques/SM, 66% de hilos
```

Información del sistema

```
NVIDIA GeForce RTX 5070 Ti, 616.56, 16303 MiB
Build cuda_12.8.r12.8/compiler.35583870_0
AMD Ryzen 7 7800X3D 8-Core Processor
Tue Nov 10 03:16:10 UTC 2026
CPU: 32.498 ms -> speedup 52.4x
```

Anexo 3. Cómo reproducir los resultados

```
make                                # compila las 3 versiones
./hough_shared runway.pgm --runs 30    # tiempos, validación e imágenes
bash scripts/run_benchmarks.sh 3 10    # bitácoras intercaladas y experimentos
bash scripts/block_sweep.sh 30         # barrido de tamaño de bloque
python scripts/compare_outputs.py      # verifica que las 3 versiones coinciden
python scripts/analyze.py              # estadísticas y tablas del informe
python scripts/plots.py                # gráficas
```

En Windows se puede usar Docker.

```
docker run --rm --gpus all -v "${PWD}:/work" -w /work \
  nvidia/cuda:12.8.1-devel-ubuntu22.04 bash -c "make && bash scripts/run_benchmarks.sh"
```

La estructura del repositorio es la siguiente.

```
src/hough_global.cu                kernel con memoria global
src/hough_const.cu                 global y constante
src/hough_shared.cu                global, constante y compartida, la versión final
src/experiments/variants.cu        experimentos de la pregunta 7a y del volcado
src/experiments/optimizaciones.cu  optimizaciones de la versión compartida
common/                            lectura PGM, CPU, validación, tiempos, rectas, PNG
scripts/                           mediciones, barrido, análisis, gráficas, comparación
results/                           bitácoras CSV, resúmenes y figuras
output/                            imágenes de salida y acumuladores
Informe_Proyecto3_Paralela.pdf     este informe
```